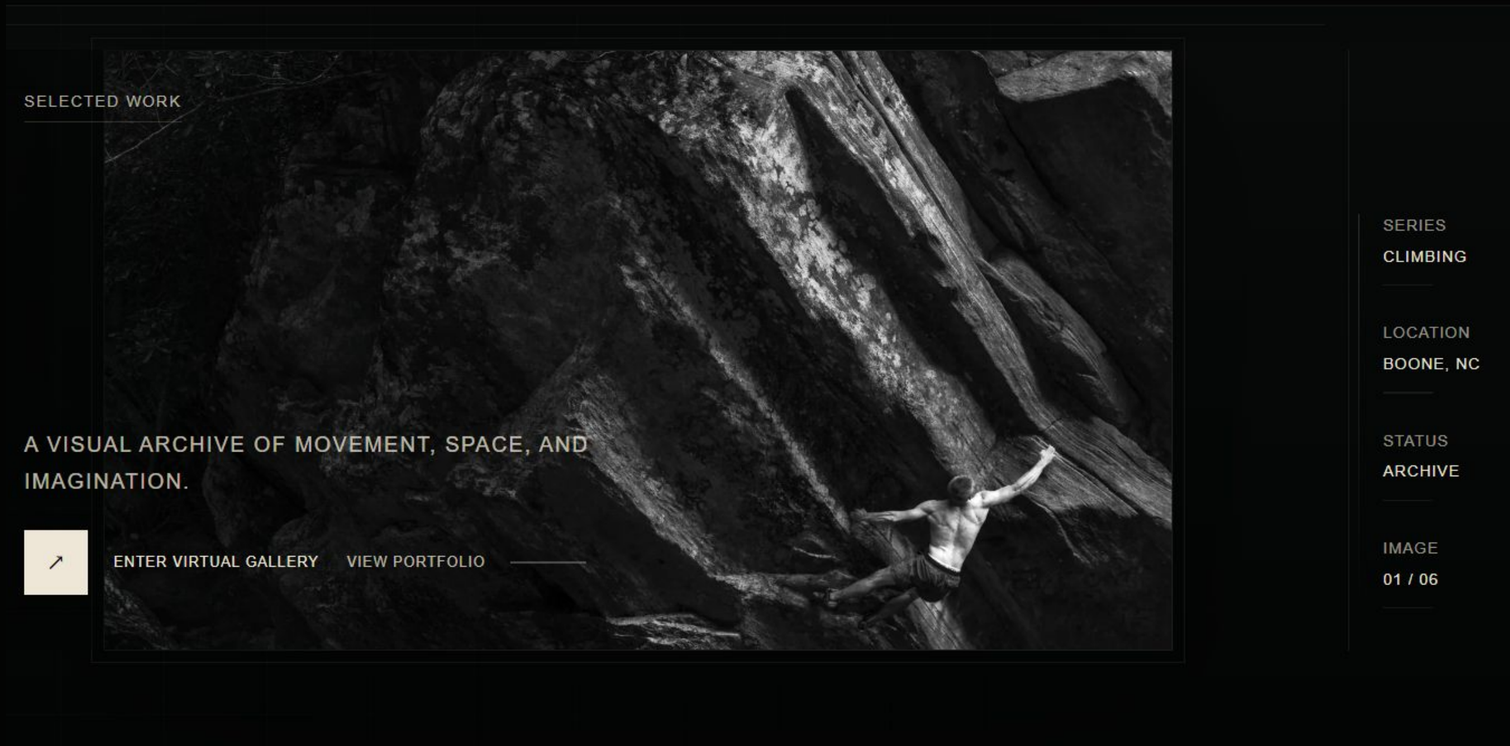
Building My Portfolio System

A local editor and an adaptive Three.js gallery



TAYLOR PIKE

One content system for the portfolio site and virtual gallery

01 / ORIGIN

Why I built a virtual gallery

STARTING POINT

I did not want my portfolio to feel like another image grid. After learning Three.js, I started testing what it would feel like to walk through the work instead of only scrolling past it.

It gave me a reason to bring photography and coding into the same project. I had always wanted to build something experimental in JavaScript since I first learned to code in high school.

WHAT CHANGED

I can use movement, scale, and sequence as part of the presentation.

02 / EVOLUTION

The gallery needed an editor

The first issue was maintenance, not rendering. I was renaming files, generating image sizes, and editing paths and JSON by hand. I knew this workflow would cause me to burn out.

Whenever the same decision came up again, I built a small interface for it instead of returning to the source code.

MY APPROACH

I built each tool around a workflow problem I was already having. I was not trying to make a generic CMS.

- 01 Virtual gallery idea
- 02 Image import tool
- 03 Homepage + portfolio curation
- 04 Metadata + crop controls
- 05 About-page composition
- 06 Gallery architecture + wall curation
- 07 Site-wide control panel

RESULT

The editor now has a few specialized workspaces for decisions that regular form fields do not handle well.

The editor became my main tool.

03 / SYSTEM

How the pieces fit together

I update the portfolio through the local editor instead of editing the site code.

AUTHORING

Local editor

Structured inputs + visual workspaces

CONTENT

Structured record

Used across the site

PRESENTATION

Traditional site

ASSETS

Purpose-specific media

EXPERIENCE

Three.js gallery

SYSTEM RULE I update content without rewriting the site. Almost all of the data on the site is built upon JSON files.

04 / INGESTION

Importing one photograph

The importer validates one source image, creates four WebP files for different uses, writes the image record, and keeps the original.

INPUT
Source image

VALIDATE
Metadata, Category, Duplicates

THUMB	TEXTURE	DISPLAY	FULL
520	**1024**	**1600**	**2400**
PX / WEBP	PX / WEBP	PX / WEBP	PX / WEBP
Library + grid	Three.js	Public display	Close viewing

OUTPUT
JSON record + backup + source archive

After import, I move the processed original into an archive. The live site only receives the image files each view needs.

05 / CONTENT

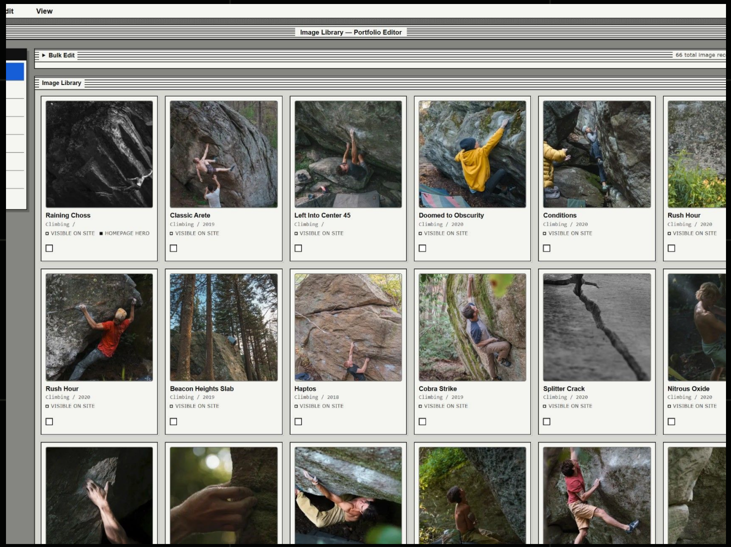
The parts I can change without code

The editor changes the data the site reads. Responsive behavior, animation, accessibility, and application logic stay in the codebase.

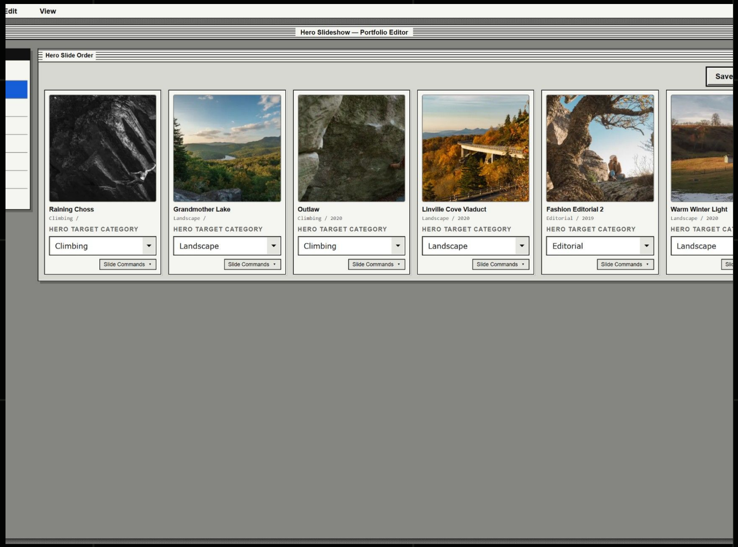
EDITABLE CONTENT

- Images + categories
- Hero slides
- About copy
- Gallery curation
- SEO + entry copy

IMAGE LIBRARY



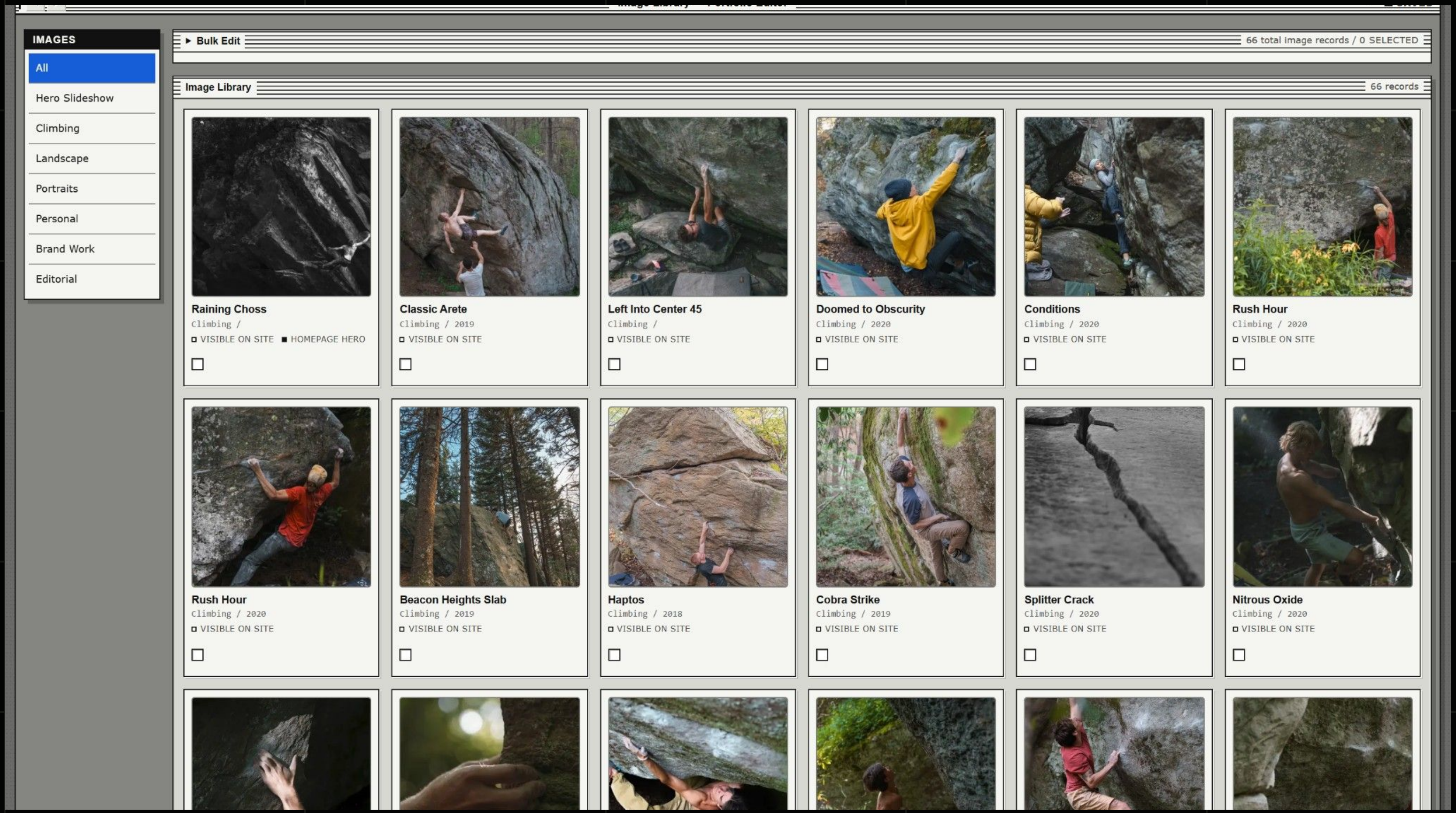
HOMEPAGE CURATION



The same image and content records can be used on the homepage, in the portfolio, and inside the gallery.

06 / EDITOR

The editor I actually use



HOW IT IS ORGANIZED

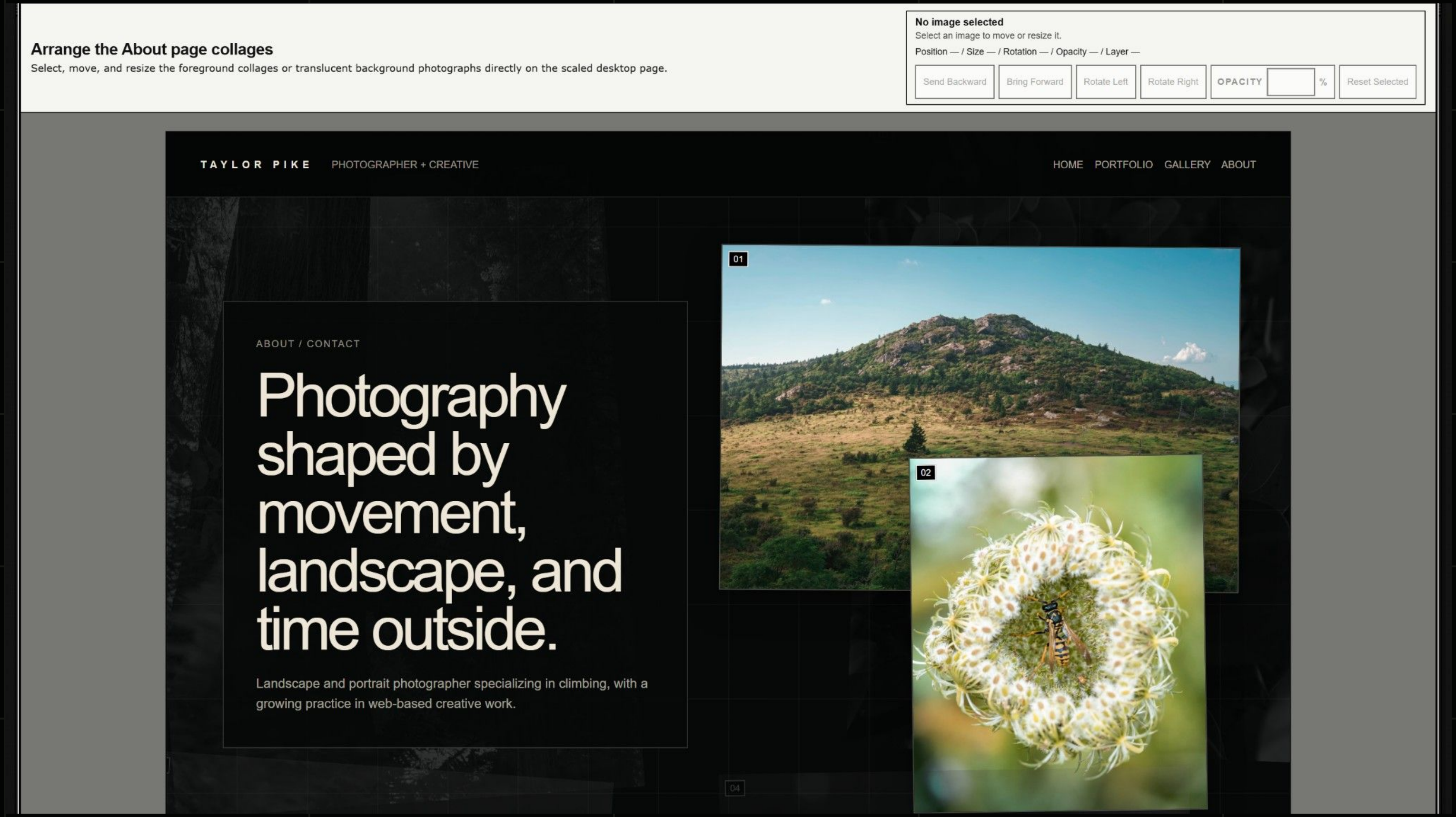
I use regular fields for titles, metadata, categories, copy, labels, SEO, and site-wide text.

For crops, ordering, the About page layout, and gallery placement, I use separate visual workspaces.

MEDIA	central library
DATA	structured fields
CURATION	site assignments
RECOVERY	save + backups

07 / COMPOSITION

I kept the visual controls narrow



For the About page, I only need controls for:

- Position + size
- Layer + rotation
- Opacity + crop

DESKTOP

The basis of the layout.

OTHER SCREENS

Tablet is a preview. Mobile can have its own arrangement.

A custom visual editing interface was made to maximize design efficiency.

08 / EXPERIENCE

Why the gallery stays separate

Main portfolio

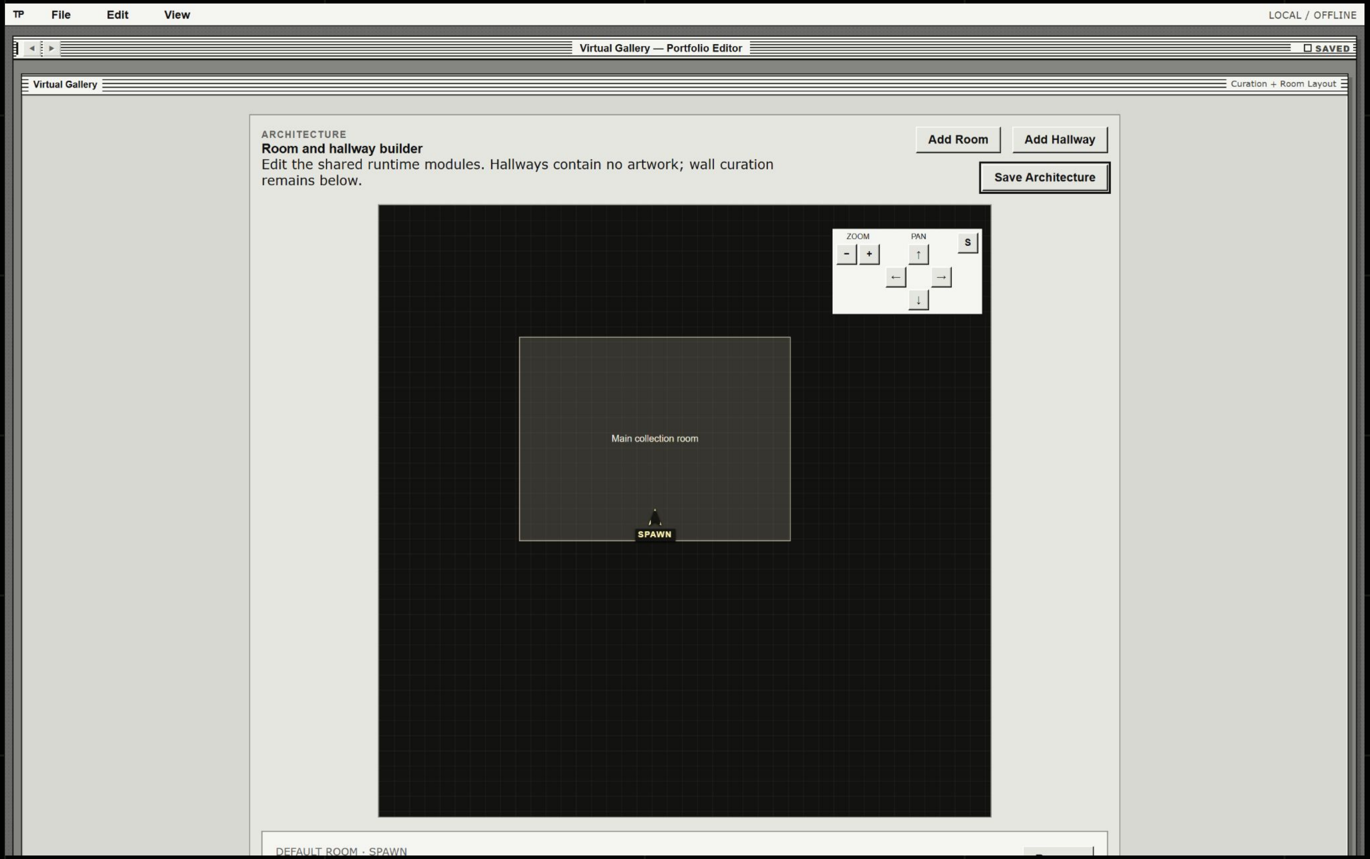
The quickest way to see the work

Virtual gallery

A slower, unique and meaningful way to move through the work

09 / ARCHITECTURE

Planning beyond the first room



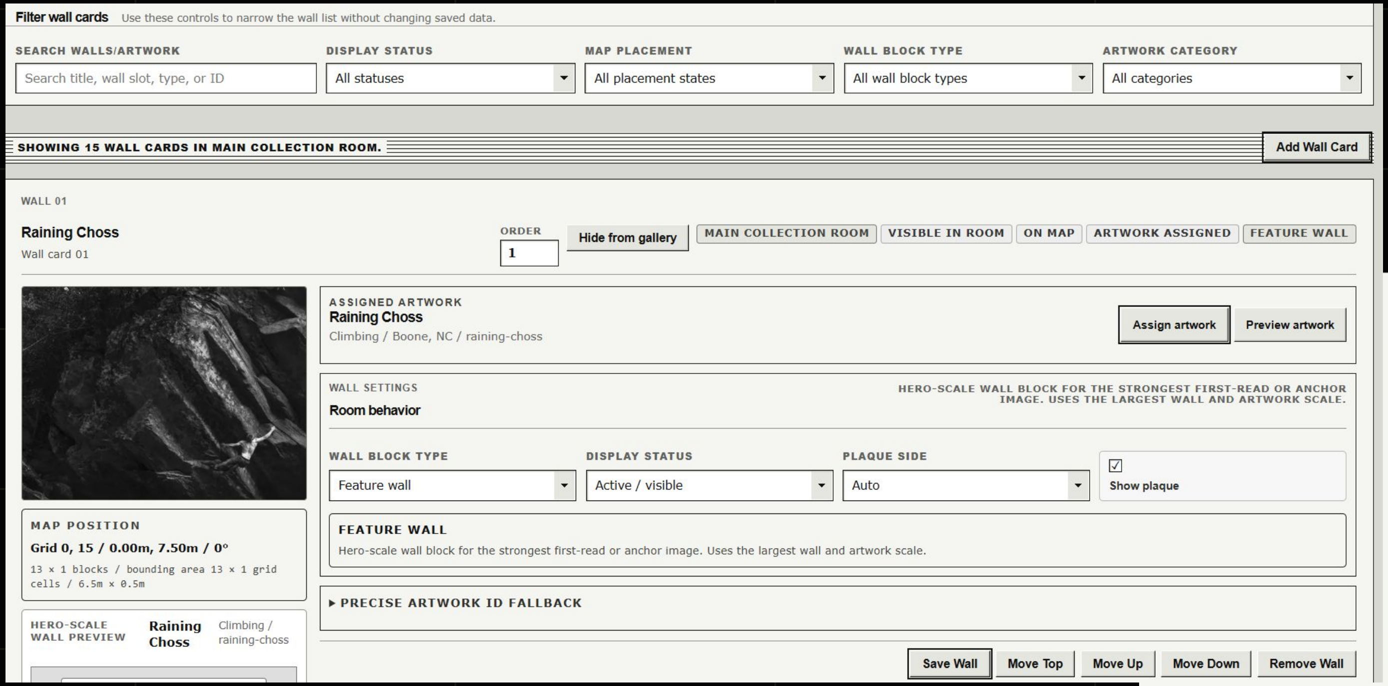
I keep rooms, hallways, walls, connections, and artwork assignments in separate records so I can add to the gallery without rebuilding it.

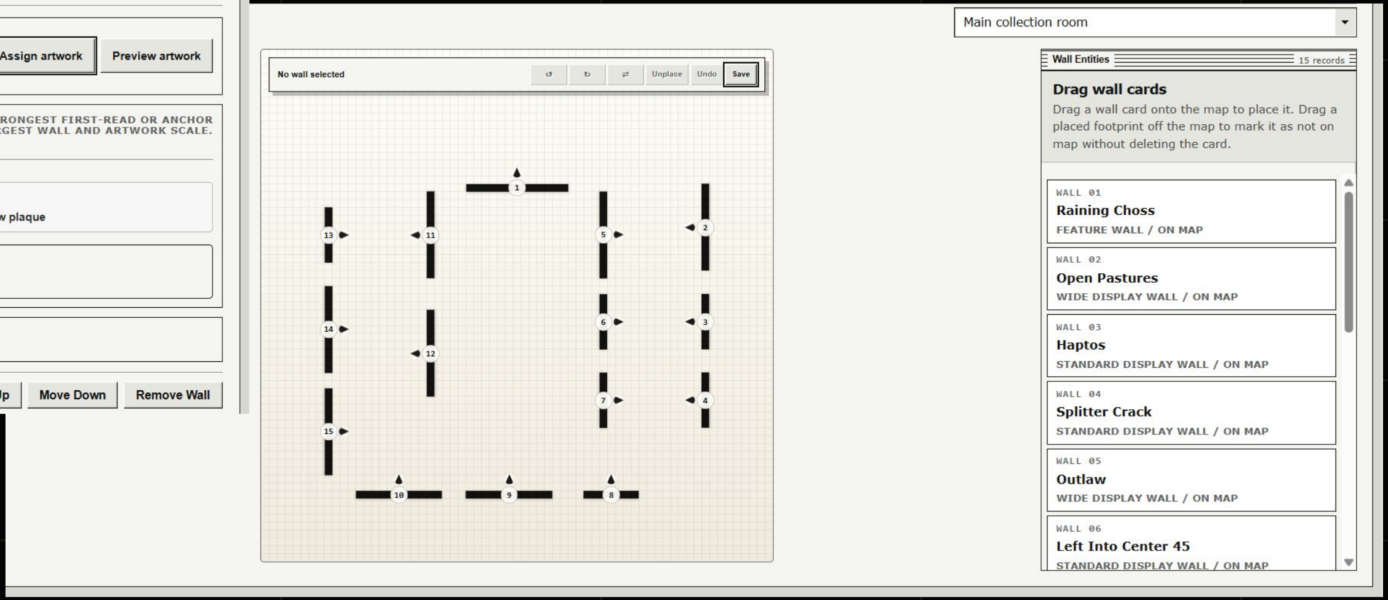
The gallery is small now, but the I created the ability to expand when needed.

Local walls
Artwork assignments
Connection points

10 / CURATION

Placing work inside the gallery





ARCHITECTURE DATA

Rooms, hallways, dimensions, connections, collision

CURATION DATA

Artwork, wall, position, scale, plaque, sequence

11 / PERFORMANCE

The gallery adjusts to the device

Auto always starts on Low. It moves to Medium only after performance stays healthy and the textures are ready. If that test fails, it stays on Low.

WHAT STAYS THE SAME

Core experience

- Room architecture
- Artwork selection
- Wall placement
- Movement + navigation
- Basic visibility

WHAT CHANGES

Rendering cost

- Artwork texture size
- Architectural lighting
- Fixture visibility
- Wall-wash behavior
- Pixel ratio + shadows

Visitor-facing quality control

AUTO Starts Low

LOW Static light; fixtures hidden

MEDIUM Pooled washes; fixtures visible

HIGH Full textures + shadows

12 / EVIDENCE

What changes between Low, Medium, and High

The room, artwork, and navigation are the same at every tier. Lighting, texture resolution, fixtures, pixel ratio, and shadows are not.

LOW

- Static architectural lighting
- Track fixtures hidden
- 0.95 pixel-ratio cap
- No shadows

MEDIUM

- Track fixtures visible
- Pooled wall washes
- 520 px artwork textures
- No shadows

HIGH

- All 23 artwork lights
- Up to 1.5 pixel ratio
- 1024 px artwork textures
- Spotlight shadows

Medium and High can look similar in screenshots. High uses more detailed textures, all of the artwork lights, and shadows.

13 / DELIVERY

Downloads and GPU memory stay separate

The browser can request images before the gallery opens. I control when they are decoded and uploaded as textures, then release the GPU textures when the visitor leaves.

01 Portfolio entry → 02 Cache warming → 03 Gallery entry → 04 Decode + upload → 05 Active textures → 06 Gallery exit → 07 Return visit

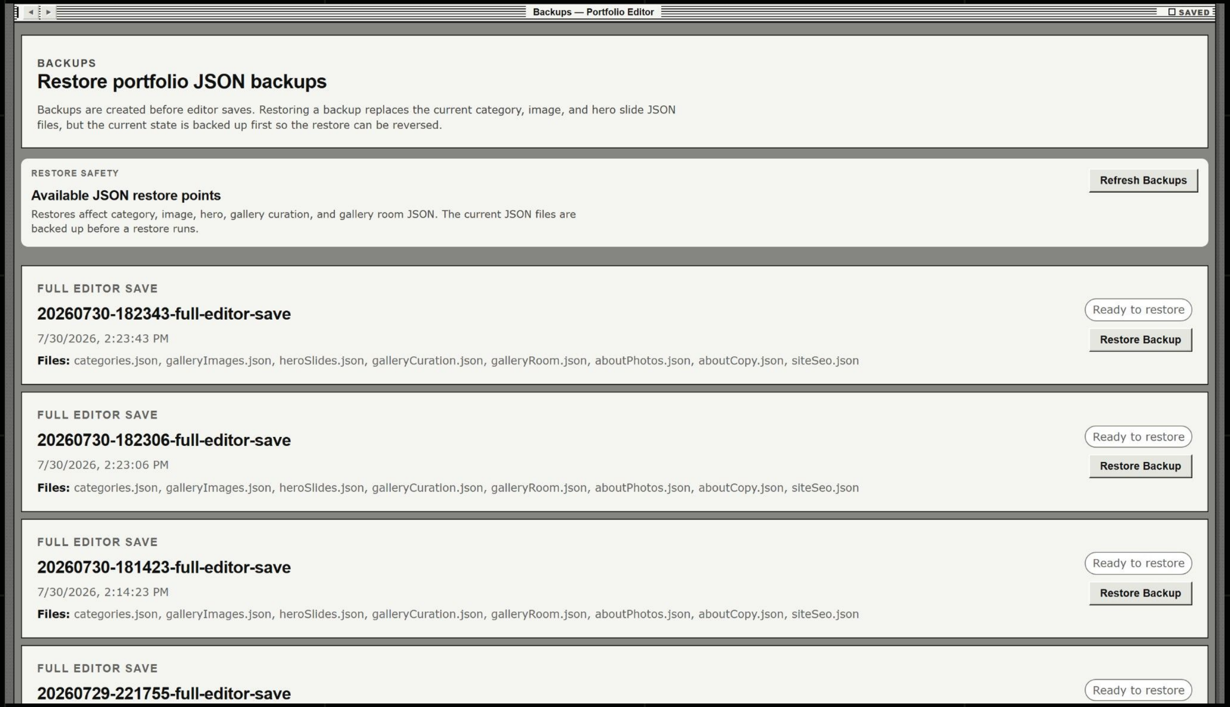
A return visit can reuse cached files without leaving the textures in GPU memory the entire time.

MEMORY	NETWORK
GPU artwork textures released	Browser cache retained

Keeping browser and GPU readiness separate makes prefetching useful without keeping GPU memory occupied.

14 / RELIABILITY

Backups, checks, and unfinished work



RELIABILITY

The editor changes the same data the public site uses, so every save creates a backup before replacing the active record. Imports validate data and archive originals. Build, data, and layout checks catch problems before release.

CURRENT LIMITATIONS

The editor still runs locally and is built specifically for this portfolio. This means any JSON changes need to be manually committed and pushed to Github. Some parts of the interface still need a more consistent design pass, and it is not a standalone product.

VALIDATE — **BACK UP** — **SAVE** — **BUILD** — **REVIEW** — **RELEASE**

15 / DIRECTION

Where I want to take the editor next

I built the current editor for one website. The next step is a schema-driven control panel that can connect to other custom sites without exposing their code.

SYSTEM

Developer-controlled

- Source code + templates
- Responsive rules
- Accessibility
- Application logic + infrastructure

PRODUCT

Custom-site control panel

Reusable structured tools

+ specialized visual workspaces

CONTENT

Owner-controlled

- Media + copy
- SEO + navigation
- Approved presets
- Bounded layout data

I started with a gallery for my own photographs. The editor grew from the work required to maintain it.